Introduction to Data Science
Final-Term Project Report

GDP Per Capita Level Classification Using World Bank Development Indicators

Data Source: World Bank Public Web Endpoints
Software: RStudio and R
Method: R-based Web Scraping / Web Data Extraction

Item	Details
Student Name	_____
Student ID	_____
Course	Introduction to Data Science
Submission Date	6 May 2026

Abstract

This project applies a complete data science workflow to World Bank development indicator data. The goal is to classify country-year records into high and low GDP per capita groups using social and economic indicators. The dataset was collected from publicly available World Bank web endpoints using R-based web scraping and web data extraction methods. The collected web data was cleaned, transformed, explored using visual analysis, and used to train two classification models: Decision Tree and Random Forest. The final cleaned dataset contains 1,051 observations and 13 variables. The Decision Tree model achieved an accuracy of 87.03%, while the Random Forest model achieved an accuracy of 96.52%. Based on these results, Random Forest was the best-performing model for this classification task.

Table of Contents

1. 1. Research Objective Definition
2. 2. Data Collection
3. 3. Data Understanding and Exploration
4. 4. Data Preprocessing
5. 5. Modeling and Analysis
6. 6. Results and Interpretation
7. 7. Conclusion and Future Work
8. 8. Project Deliverables
9. Appendix A: Generated Files
10. Appendix B: Project Requirement Check
11. Appendix C: Source Code

1. Research Objective Definition

The main objective of this project is to predict whether a country-year observation belongs to a high GDP per capita group or a low GDP per capita group. The prediction is performed using development indicators such as population, life expectancy, percentage of internet users, urban population percentage, unemployment rate, region, and income group. The problem is framed as a binary classification task, where the target variable is high_gdp_per_capita.

Component	Description
Research type	Classification
Target variable	high_gdp_per_capita
Target definition	1 if GDP per capita is greater than or equal to the median value, otherwise 0
Prediction goal	Classify country-year records as high or low GDP per capita
Time period	2018 to 2022

2. Data Collection

The dataset was collected from publicly available World Bank web sources using RStudio. The World Bank GDP per capita source page was accessed with the rvest package to document the public web source. Since the World Bank website exposes large country-year indicator records through public web endpoints, the project generated endpoint URLs for each selected indicator and year range. The httr package was used to send HTTP GET requests to those URLs, and httr::content() was used to retrieve the returned web content. The jsonlite package parsed the returned content into R data frames. Each indicator was collected separately and then merged using country and year as common keys with dplyr and tidyr. This R-based web scraping and web data extraction approach was used because it collects the data programmatically from a public source, produces a reproducible dataset, and avoids manual downloading.

2.1 Indicators Used

Variable	World Bank Indicator Code	Description
GDP per capita	NY.GDP.PCAP.CD	GDP per capita in current US dollars; used to create the target variable
Population	SP.POP.TOTL	Total population
Life expectancy	SP.DYN.LE00.IN	Life expectancy at birth, total years
Internet users	IT.NET.USER.ZS	Individuals using the Internet as a percentage of population
Urban population percentage	SP.URB.TOTL.IN.ZS	Urban population as a percentage of total population
Unemployment rate	SL.UEM.TOTL.ZS	Unemployment as a percentage of total labor force

2.2 Dataset Size

Item	Value
Raw scraped/collected dataset	1,330 rows and 18 columns
Final cleaned dataset	1,051 rows and 13 columns
Decision Tree model dataset	1,051 rows and 7 columns
Random Forest model dataset	1,051 rows and 9 columns
Target class 0	525 observations
Target class 1	526 observations

3. Data Understanding and Exploration

The final dataset contains both descriptive variables and numeric development indicators. Country name, country code, region, and income group were retained for interpretation and reporting. Numeric variables were used for exploratory analysis and modeling. The dataset represents country-year observations between 2018 and 2022.

3.1 Dataset Variables

Column	Data Type	Description
country	Character	Country name
iso2c	Character	Two-letter country code
year	Integer	Observation year
region	Character	World Bank region
income	Character	World Bank income group
gdp_per_capita	Numeric	GDP per capita in current US dollars
population	Numeric	Total population
life_expectancy	Numeric	Life expectancy in years
internet_users	Numeric	Internet users percentage
urban_population_pct	Numeric	Urban population percentage
unemployment_rate	Numeric	Unemployment rate percentage
log_population	Numeric	Log-transformed population
high_gdp_per_capita	Numeric/Binary	Classification target, 0 or 1

3.2 Sample of Final Dataset

The table below shows a sample of the final cleaned dataset. The complete dataset is submitted separately as a CSV file.

country	iso2c	year	income	gdp_per_capita	population	life_expectancy	internet_users	urban_population_pct	unemployment_rate	high_gdp_per_capita
Afghanistan	AF	2018	Low income	491.34	36,743,039	62.44	16.80	25.00	11.19	0
Afghanistan	AF	2019	Low income	496.60	37,856,121	62.94	17.60	25.14	11.19	0
Afghanistan	AF	2020	Low income	510.79	39,068,979	61.45	17.05	25.26	11.71	0
Afghanistan	AF	2021	Low income	356.50	40,000,412	60.42	16.51	25.35	12.01	0
Afghanistan	AF	2022	Low income	357.26	40,578,842	65.62	15.87	25.39	14.10	0
Albania	AL	2018	Upper middle income	5,897.65	2,607,733	79.24	65.40	56.38	12.30	0
Albania	AL	2019	Upper middle income	6,069.44	2,567,801	79.47	68.55	56.76	11.47	0
Albania	AL	2020	Upper middle income	6,027.91	2,528,480	77.82	72.24	57.14	11.64	0

3.3 Summary Statistics

Variable	Min	1st Qu.	Median	Mean	3rd Qu.	Max
GDP per capita	234.31	2,531.23	7,037.01	19,485.20	25,851.11	226,052.00

Population	9,992.00	811,573.50	6,565,267.00	36,872,590.38	25,420,460.50	1,425,423,212.00
Life expectancy	18.82	67.11	73.50	72.69	78.26	86.15
Internet users	2.73	49.20	71.11	65.07	83.56	100.00
Urban population pct	13.71	42.68	63.38	61.96	81.16	100.00
Unemployment rate	0.10	4.01	5.59	7.37	9.01	36.47
Log population	4.00	5.91	6.82	6.63	7.41	9.15

3.4 Missing Values Before Cleaning

Column	Missing Count
country	0
iso2c	0
year	0
region	0
income	0
gdp_per_capita	34
population	0
life_expectancy	0
internet_users	168
urban_population_pct	0
unemployment_rate	151

3.5 Outliers, Noise, and Inconsistencies

The numeric variables showed expected real-world variation. GDP per capita and population were highly skewed because countries differ greatly in economic size and population size. These extreme values were retained because they represent valid country-level observations rather than data-entry errors. Aggregate rows such as regional or global totals were treated as inconsistencies for this country-level study and removed. Missing values were handled through row removal for GDP per capita and median imputation for selected predictor variables.

3.6 Exploratory Data Analysis Plots

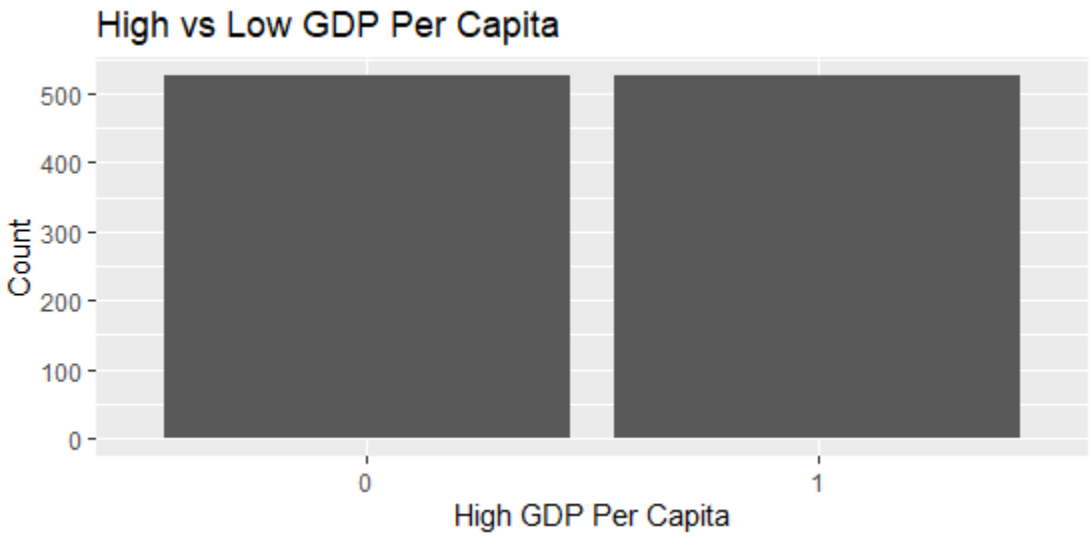


Figure 1: High vs Low GDP Per Capita

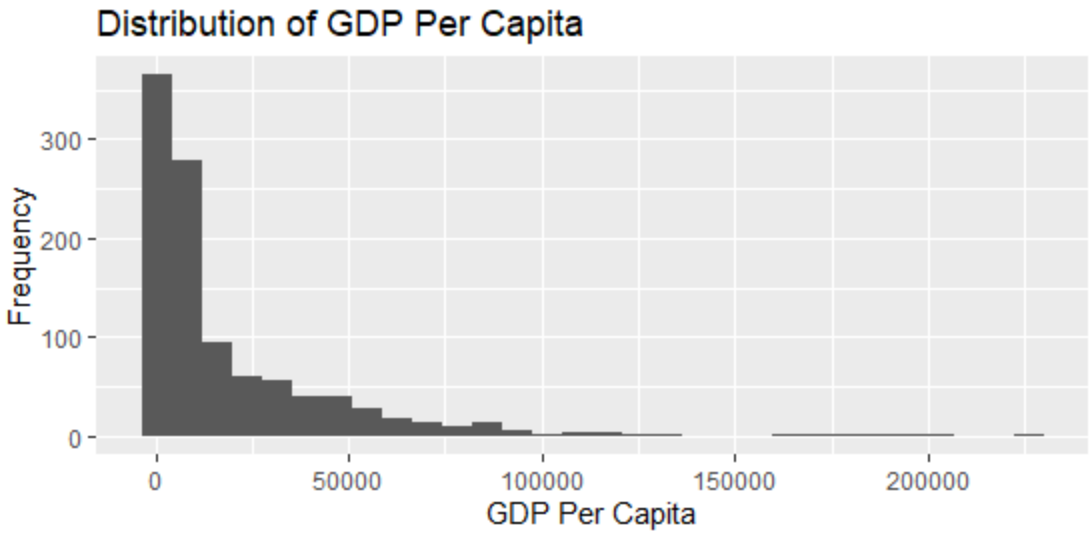


Figure 2: Distribution of GDP Per Capita

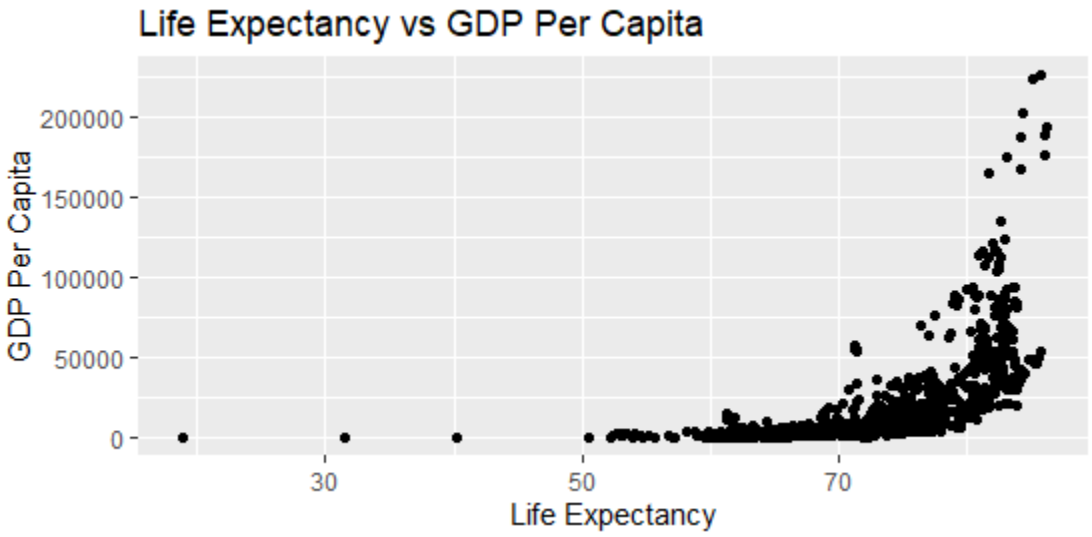


Figure 3: Life Expectancy vs GDP Per Capita

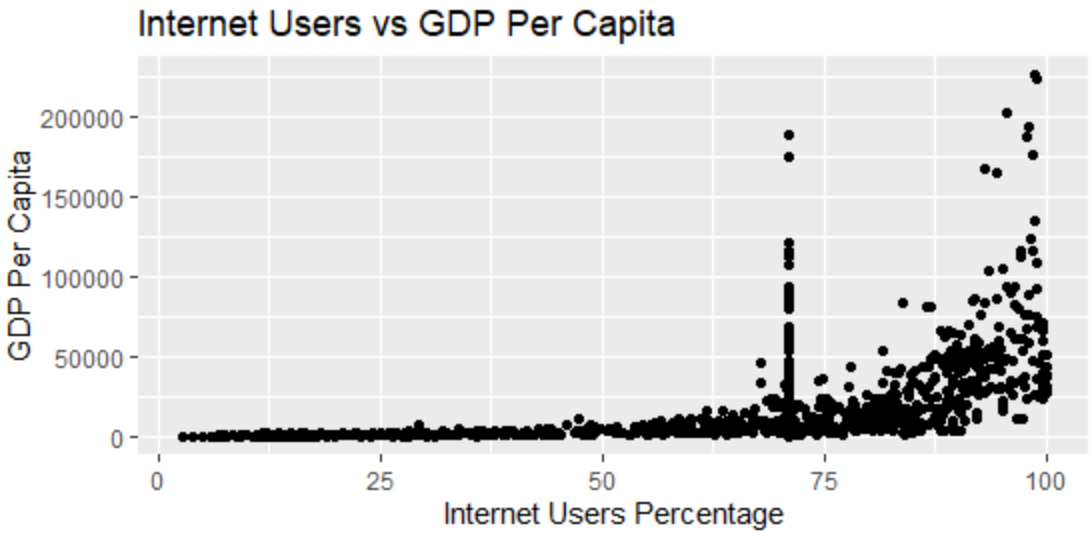


Figure 4: Internet Users vs GDP Per Capita

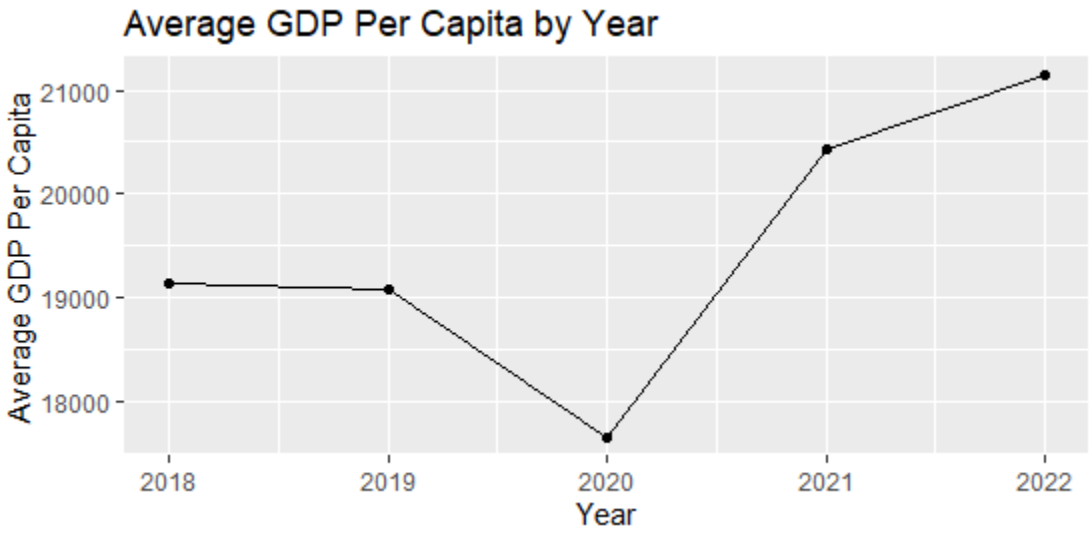


Figure 5: Average GDP Per Capita by Year

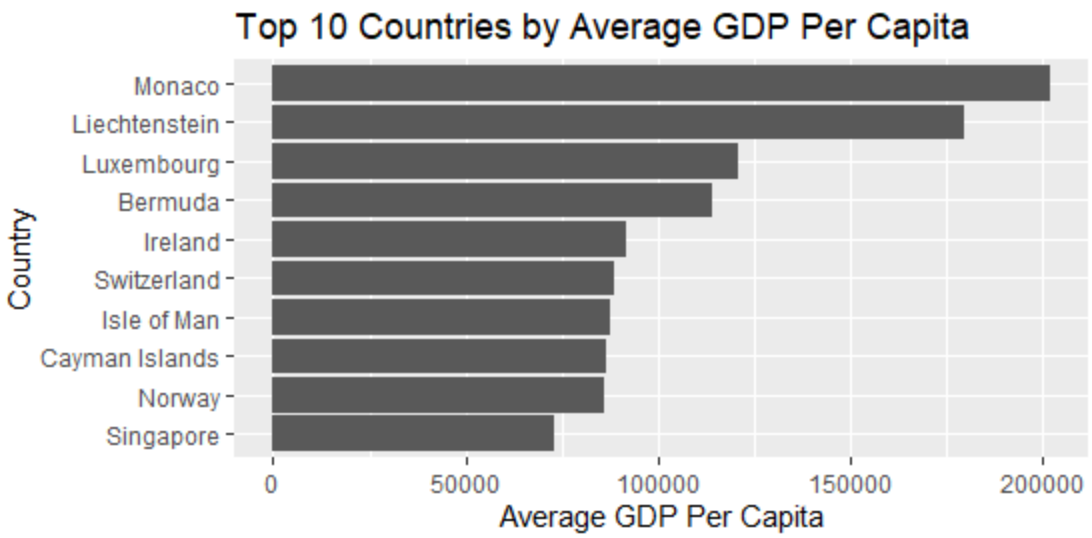


Figure 6: Top 10 Countries by Average GDP Per Capita

4. Data Preprocessing

Several preprocessing steps were applied before modeling. First, aggregate regions were removed because the study focuses on countries rather than regional or global totals. Rows with missing GDP per capita values were removed, since GDP per capita was required to create the target variable. Missing values in internet users and unemployment rate were replaced using median imputation. The population variable was transformed using log10(population + 1) to reduce the effect of very large population values. Region and income group were converted to factor variables for the Random Forest model. Standard normalization was not required because both final models were tree-based models.

Preprocessing Step	Explanation
Removed aggregate regions	Rows where region was Aggregates were excluded
Handled missing GDP	Rows with missing GDP per capita were removed
Handled missing indicators	Median imputation was applied to internet users, unemployment rate, and other numeric indicators when needed
Feature engineering	log_population was created from population
Categorical encoding	region and income were converted to factors for the Random Forest model
Scaling/normalization	Not required because Decision Tree and Random Forest are tree-based models
Target engineering	high_gdp_per_capita was created using median GDP per capita

4.1 Model Datasets

The Decision Tree model used numeric predictors only. The Random Forest model used the same numeric predictors plus region and income group to improve predictive performance.

Model	Features Used
Decision Tree	year, log_population, life_expectancy, internet_users, urban_population_pct, unemployment_rate
Random Forest	year, region, income, log_population, life_expectancy, internet_users, urban_population_pct, unemployment_rate

5. Modeling and Analysis

The data was split into training and testing sets using a 70:30 ratio. Two classification models were implemented: Decision Tree and Random Forest. The Decision Tree model was selected because it is interpretable and can model non-linear decision rules. Random Forest was selected because it builds an ensemble of multiple decision trees and usually provides stronger predictive performance than a single tree. GDP per capita was not used as an input feature in either model because it was used to create the target variable.

Item	Details
Training set	70% of observations
Testing set	30% of observations
Model 1	Decision Tree Classification using rpart
Model 2	Random Forest Classification using randomForest
Evaluation metrics	Confusion matrix, accuracy, precision, recall, and F1-score

5.1 Model Features

Feature	Description
year	Observation year
region	World Bank region; used in Random Forest
income	World Bank income group; used in Random Forest
log_population	Log-transformed population
life_expectancy	Life expectancy at birth
internet_users	Internet users percentage
urban_population_pct	Urban population percentage
unemployment_rate	Unemployment rate

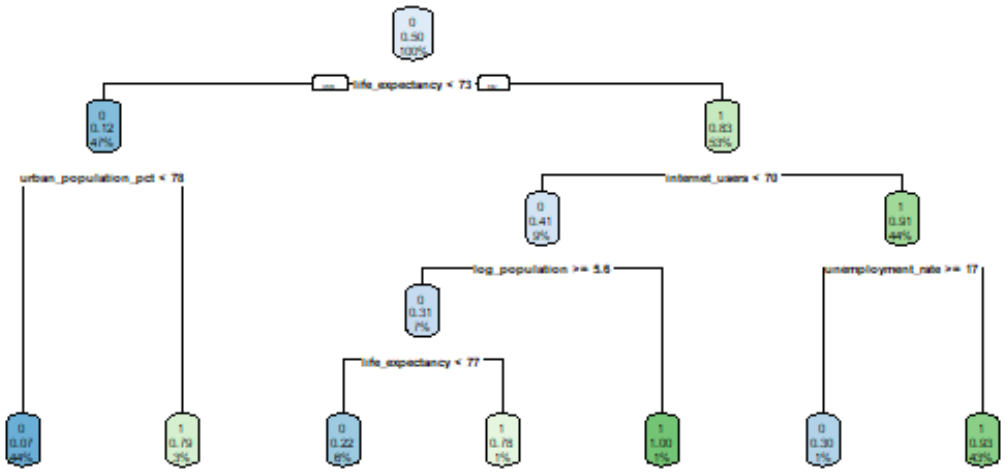


Figure 7: Decision Tree Classification Model

6. Results and Interpretation

The Random Forest model produced the highest accuracy among the two models. The target variable was well balanced, with 525 low GDP per capita observations and 526 high GDP per capita observations. Therefore, accuracy is a reasonable first evaluation measure for this project.

6.1 Target Distribution

Class	Count
0: Low GDP per capita	525
1: High GDP per capita	526

6.2 Model Performance

Model	Accuracy	Accuracy (%)	Precision	Recall	F1-score
Decision Tree	0.8703	87.03%	0.8790	0.8625	0.8707
Random Forest	0.9652	96.52%	0.9745	0.9563	0.9653

6.3 Decision Tree Confusion Matrix

Actual Class	Predicted 0	Predicted 1
Actual 0	137	19
Actual 1	22	138

For the Decision Tree model, 137 low GDP observations and 138 high GDP observations were correctly classified. The model misclassified 19 low GDP observations as high GDP and 22 high GDP observations as low GDP. The overall accuracy was 87.03%.

6.4 Random Forest Confusion Matrix

Actual Class	Predicted 0	Predicted 1
Actual 0	152	4
Actual 1	7	153

For the Random Forest model, 152 low GDP observations and 153 high GDP observations were correctly classified. The model misclassified 4 low GDP observations as high GDP and 7 high GDP observations as low GDP. The overall accuracy was 96.52%.

6.5 Random Forest Interpretation

Random Forest improved performance by combining many decision trees and reducing the instability of a single decision tree. It also allowed the model to use categorical indicators such as region and income group along with numeric indicators. The strong accuracy suggests that GDP per capita classification is strongly associated with life expectancy, internet usage, urban population percentage, unemployment rate, population, region, and income group.

6.6 Best Performing Model

The Random Forest model was selected as the best performing model because it achieved the highest accuracy, 96.52%, compared with 87.03% for the Decision Tree. The Random Forest model also produced higher precision, recall, and F1-score, meaning it was better at identifying both high and low GDP per capita observations.

7. Conclusion and Future Work

This project successfully collected real-world development data from publicly available World Bank web endpoints using R-based web scraping and web data extraction. The dataset was cleaned, transformed, explored through visual analysis, and used to train classification models. Both Decision Tree and Random Forest performed well, but Random

Forest gave the best accuracy. The results indicate that life expectancy, internet usage, urban population percentage, unemployment rate, population, region, and income group are useful indicators for classifying GDP per capita level.

7.1 Limitations

- Only data from 2018 to 2022 was used, so long-term economic trends were not fully captured.
- GDP per capita was converted into a binary target using the median, which simplifies a continuous economic measure.
- Some indicators had missing values and required median imputation.
- The CO2 emissions indicator was not included in the final scraping workflow to keep the dataset stable and reproducible.
- The analysis is based on country-year records and does not capture all political, social, and institutional factors affecting GDP.

7.2 Future Work

- Include additional indicators such as education, health expenditure, inflation, trade, energy use, and governance indicators.
- Use a longer time period to analyze trends over decades.
- Compare more models such as Support Vector Machine or Gradient Boosting.
- Use regression modeling to predict actual GDP per capita instead of only classifying high and low GDP groups.
- Apply cross-validation for more robust performance estimation.

8. Project Deliverables

Deliverable	Description
Final report	This Word document
Source code	R script used for web scraping/data extraction, preprocessing, EDA, and modeling
Final dataset	world_bank_gdp_project_dataset.csv
Model results	world_bank_model_results.csv
Confusion matrices	decision_tree_confusion_matrix.csv and random_forest_confusion_matrix.csv
Plots	Seven plot images generated and saved in the plots folder

References

- World Bank. World Bank Data. https://data.worldbank.org/
- R Core Team. R: A Language and Environment for Statistical Computing. https://www.r-project.org/
- R packages used for web data collection and analysis: rvest, httr, jsonlite, dplyr, tidyr, ggplot2, rpart, rpart.plot, and randomForest.
- Therneau, T. and Atkinson, B. rpart: Recursive Partitioning and Regression Trees.
- Liaw, A. and Wiener, M. randomForest: Breiman and Cutler's Random Forests for Classification and Regression.

Appendix A: Generated Files

File Name	Purpose
source_information.csv	Records the public World Bank source page and collection method
world_bank_gdp_project_dataset.csv	Final cleaned dataset created from scraped World Bank web data with 1,051 rows and 13 columns
missing_values_before_cleaning.csv	Missing-value counts before cleaning

world_bank_model_results.csv	Accuracy, precision, recall, and F1-score for Decision Tree and Random Forest
decision_tree_confusion_matrix.csv	Confusion matrix for Decision Tree
random_forest_confusion_matrix.csv	Confusion matrix for Random Forest
random_forest_feature_importance.csv	Feature importance values generated by the Random Forest model
plots/01_high_low_gdp_count.png	High vs Low GDP Per Capita count plot
plots/02_gdp_distribution.png	GDP Per Capita distribution plot
plots/03_life_expectancy_vs_gdp.png	Life Expectancy vs GDP Per Capita scatter plot
plots/04_internet_users_vs_gdp.png	Internet Users vs GDP Per Capita scatter plot
plots/05_average_gdp_by_year.png	Average GDP Per Capita by Year plot
plots/06_top_10_gdp_countries.png	Top 10 Countries by Average GDP Per Capita plot
plots/07_decision_tree_model.png	Decision Tree model plot

Appendix B: Project Requirement Check

The following checklist verifies that the report follows the final-term project requirements. The project includes public web data collection, data understanding, preprocessing, modeling, results interpretation, conclusion, deliverables, and source code.

Requirement	Status	Evidence in Report
Research objective definition	Complete	Section 1 defines a classification objective: predicting high vs low GDP per capita.
Data collection from public source	Complete	Section 2 explains World Bank public web endpoint data collection using R-based web scraping/data extraction with rvest, httr, and jsonlite.
Dataset structure and understanding	Complete	Section 3 describes variables, data types, dataset size, sample rows, and summary statistics.
Exploratory data analysis	Complete	Section 3 includes bar charts, histogram, scatter plots, time trend, and top-country chart.
Missing values and inconsistencies	Complete	Missing values are identified and handled using row removal and median imputation.
Data preprocessing	Complete	Section 4 covers aggregate-region removal, missing-value handling, feature engineering, categorical encoding, and target creation.
Modeling and analysis	Complete	Section 5 trains Decision Tree and Random Forest using a 70:30 train-test split.
Performance evaluation	Complete	Section 6 reports confusion matrices, accuracy, precision, recall, and F1-score.
Results interpretation	Complete	Section 6 interprets model performance and identifies Random Forest as best.
Conclusion and future work	Complete	Section 7 summarizes findings, limitations, and future improvements.
Project deliverables	Complete	Section 8 lists report, source code, final dataset, model results, and plots.
Source code	Complete	Appendix C includes the complete updated R source code using rvest, httr, and jsonlite for web data collection.

Appendix C: Source Code

The following source code shows the full R script used for data collection, preprocessing, exploratory analysis, modeling, evaluation, and output generation. The same code should also be submitted as an R script file.

```
# ============================================================
# IDS Final Project
# Topic: GDP Per Capita Level Classification Using World Bank Data
# Method: R-based Web Scraping / Web Data Extraction
# Source: World Bank Public Web Endpoints
# Models: Decision Tree and Random Forest
# ============================================================

# --------------------------------
# Step 1: Install and load packages
# --------------------------------

options(timeout = 300)

packages <- c(
  "rvest",
  "httr",
```

```
  "jsonlite",
  "dplyr",
  "tidyr",
  "ggplot2",
  "rpart",
  "rpart.plot",
  "randomForest"
)

for (pkg in packages) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg, dependencies = TRUE, repos = "https://cloud.r-project.org")
    library(pkg, character.only = TRUE)
  }
}

library(rvest)
library(httr)
library(jsonlite)
library(dplyr)
library(tidyr)
library(ggplot2)
library(rpart)
library(rpart.plot)
library(randomForest)

# --------------------------------
# Step 2: Create folders
# --------------------------------

if (!dir.exists("dataset")) {
  dir.create("dataset")
}

if (!dir.exists("plots")) {
  dir.create("plots")
}

# --------------------------------
# Step 3: Read source webpage using rvest
# --------------------------------

source_url <- "https://data.worldbank.org/indicator/NY.GDP.PCAP.CD"

page_title <- tryCatch(
  {
    page <- rvest::read_html(source_url)
    page %>%
      rvest::html_node("title") %>%
      rvest::html_text()
  },
  error = function(e) {
    "World Bank GDP per capita source page"
  }
)

print(page_title)

source_info <- data.frame(
  Source_Name = "World Bank Data",
  Source_URL = source_url,
  Page_Title = page_title,
  Collection_Method = "R-based web scraping and web data extraction using rvest, httr, and jsonlite",
  stringsAsFactors = FALSE
)

write.csv(
  source_info,
  "dataset/source_information.csv",
  row.names = FALSE
)

# --------------------------------
# Step 4: Define World Bank indicators
# --------------------------------
```

```r
indicators <- c(
  gdp_per_capita = "NY.GDP.PCAP.CD",
  population = "SP.POP.TOTL",
  life_expectancy = "SP.DYN.LE00.IN",
  internet_users = "IT.NET.USER.ZS",
  urban_population_pct = "SP.URB.TOTL.IN.ZS",
  unemployment_rate = "SL.UEM.TOTL.ZS"
)

start_year <- 2018
end_year <- 2022

# --------------------------------
# Step 5: Helper functions
# --------------------------------

get_col <- function(df, col_name, default_value = NA) {
  if (col_name %in% names(df)) {
    return(df[[col_name]])
  } else {
    return(rep(default_value, nrow(df)))
  }
}

get_first_available_col <- function(df, col_names, default_value = NA) {
  for (col_name in col_names) {
    if (col_name %in% names(df)) {
      return(df[[col_name]])
    }
  }
  return(rep(default_value, nrow(df)))
}

safe_median_impute <- function(x) {
  if (all(is.na(x))) {
    return(rep(0, length(x)))
  }
  x[is.na(x)] <- median(x, na.rm = TRUE)
  return(x)
}

# --------------------------------
# Step 6: Scrape one indicator from World Bank public web endpoint
# --------------------------------

scrape_indicator <- function(indicator_code, variable_name, start_year, end_year) {

  url <- paste0(
    "https://api.worldbank.org/v2/country/all/indicator/",
    indicator_code,
    "?format=json&date=",
    start_year,
    ":",
    end_year,
    "&per_page=20000"
  )

  print(paste("Collecting:", variable_name))
  print(url)

  response <- tryCatch(
    {
      httr::GET(
        url,
        httr::user_agent("Mozilla/5.0 Data Science Student Project")
      )
    },
    error = function(e) {
      warning(paste("Request failed for:", variable_name))
      return(NULL)
    }
  )

  if (is.null(response)) {
```

```
    return(data.frame())
  }

  if (httr::status_code(response) != 200) {
    warning(paste("Failed to collect:", variable_name))
    return(data.frame())
  }

  text_data <- httr::content(
    response,
    as = "text",
    encoding = "UTF-8"
  )

  parsed_data <- tryCatch(
    {
      jsonlite::fromJSON(text_data, flatten = TRUE)
    },
    error = function(e) {
      warning(paste("JSON parsing failed for:", variable_name))
      return(NULL)
    }
  )

  if (is.null(parsed_data) || length(parsed_data) < 2 || is.null(parsed_data[[2]])) {
    warning(paste("No data found for:", variable_name))
    return(data.frame())
  }

  df <- as.data.frame(parsed_data[[2]])

  if (nrow(df) == 0) {
    warning(paste("Empty data for:", variable_name))
    return(data.frame())
  }

  result <- data.frame(
    country = as.character(get_col(df, "country.value", "")),
    iso2c = as.character(get_col(df, "country.id", "")),
    iso3c = as.character(get_col(df, "countryiso3code", "")),
    year = as.integer(get_col(df, "date", NA)),
    variable = variable_name,
    value = as.numeric(get_col(df, "value", NA)),
    stringsAsFactors = FALSE
  )

  result <- result %>%
    dplyr::filter(
      country != "",
      iso2c != "",
      !is.na(year)
    )

  Sys.sleep(0.5)

  return(result)
}

# --------------------------------
# Step 7: Collect all indicators
# --------------------------------

all_data <- list()

for (indicator_name in names(indicators)) {
  all_data[[indicator_name]] <- scrape_indicator(
    indicator_code = indicators[[indicator_name]],
    variable_name = indicator_name,
    start_year = start_year,
    end_year = end_year
  )
}

long_data <- dplyr::bind_rows(all_data)
```

```
if (nrow(long_data) == 0) {
  stop("No data was collected. Check internet connection or World Bank server.")
}

# --------------------------------
# Step 8: Convert long data to wide format
# --------------------------------

world_data <- long_data %>%
  dplyr::group_by(country, iso2c, iso3c, year, variable) %>%
  dplyr::summarise(value = mean(value, na.rm = TRUE), .groups = "drop") %>%
  tidyr::pivot_wider(
    names_from = variable,
    values_from = value
  )

# --------------------------------
# Step 9: Scrape country metadata
# --------------------------------

scrape_country_metadata <- function() {

  url <- "https://api.worldbank.org/v2/country/all?format=json&per_page=400"

  print("Collecting country metadata")
  print(url)

  response <- tryCatch(
    {
      httr::GET(
        url,
        httr::user_agent("Mozilla/5.0 Data Science Student Project")
      )
    },
    error = function(e) {
      warning("Country metadata request failed.")
      return(NULL)
    }
  )

  if (is.null(response)) {
    return(data.frame())
  }

  if (httr::status_code(response) != 200) {
    warning("Country metadata could not be collected.")
    return(data.frame())
  }

  text_data <- httr::content(
    response,
    as = "text",
    encoding = "UTF-8"
  )

  parsed_data <- tryCatch(
    {
      jsonlite::fromJSON(text_data, flatten = TRUE)
    },
    error = function(e) {
      warning("Country metadata JSON parsing failed.")
      return(NULL)
    }
  )

  if (is.null(parsed_data) || length(parsed_data) < 2 || is.null(parsed_data[[2]])) {
    warning("No country metadata found.")
    return(data.frame())
  }

  df <- as.data.frame(parsed_data[[2]])

  metadata <- data.frame(
    iso2c = as.character(get_first_available_col(df, c("iso2Code", "id"), "")),
    country_name_meta = as.character(get_first_available_col(df, c("name"), "")),
```

```
    region = as.character(get_first_available_col(df, c("region.value"), "")),
    income = as.character(get_first_available_col(df, c("incomeLevel.value"), "")),
    capital = as.character(get_first_available_col(df, c("capitalCity"), "")),
    longitude = as.character(get_first_available_col(df, c("longitude"), "")),
    latitude = as.character(get_first_available_col(df, c("latitude"), "")),
    stringsAsFactors = FALSE
  )

  return(metadata)
}

country_metadata <- scrape_country_metadata()

# --------------------------------
# Step 10: Merge indicator data with country metadata
# --------------------------------

if (nrow(country_metadata) > 0) {
  world_data <- world_data %>%
    dplyr::left_join(country_metadata, by = "iso2c") %>%
    dplyr::mutate(
      country = ifelse(
        !is.na(country_name_meta) & country_name_meta != "",
        country_name_meta,
        country
      )
    )
} else {
  world_data$region <- NA
  world_data$income <- NA
}

# --------------------------------
# Step 11: Remove aggregate regions
# --------------------------------

if ("region" %in% names(world_data)) {
  world_data <- world_data %>%
    dplyr::filter(region != "Aggregates")
}

# --------------------------------
# Step 12: Select useful columns
# --------------------------------

world_data <- world_data %>%
  dplyr::select(
    country,
    iso2c,
    year,
    region,
    income,
    gdp_per_capita,
    population,
    life_expectancy,
    internet_users,
    urban_population_pct,
    unemployment_rate
  )

# --------------------------------
# Step 13: View raw collected data
# --------------------------------

print(head(world_data))
print(dim(world_data))
str(world_data)

# --------------------------------
# Step 14: Check missing values
# --------------------------------

missing_values <- colSums(is.na(world_data))
print(missing_values)
```

```r
missing_values_df <- data.frame(
  Column = names(missing_values),
  Missing_Count = as.numeric(missing_values)
)

write.csv(
  missing_values_df,
  "dataset/missing_values_before_cleaning.csv",
  row.names = FALSE
)

# --------------------------------
# Step 15: Clean dataset
# --------------------------------

world_data_clean <- world_data %>%
  dplyr::filter(
    !is.na(gdp_per_capita),
    !is.na(population)
  )

# --------------------------------
# Step 16: Handle missing values
# --------------------------------

world_data_clean$life_expectancy <- safe_median_impute(world_data_clean$life_expectancy)
world_data_clean$internet_users <- safe_median_impute(world_data_clean$internet_users)
world_data_clean$urban_population_pct <- safe_median_impute(world_data_clean$urban_population_pct)
world_data_clean$unemployment_rate <- safe_median_impute(world_data_clean$unemployment_rate)

world_data_clean$region[is.na(world_data_clean$region) | world_data_clean$region == ""] <- "Unknown"
world_data_clean$income[is.na(world_data_clean$income) | world_data_clean$income == ""] <- "Unknown"

# --------------------------------
# Step 17: Feature engineering
# --------------------------------

world_data_clean$log_population <- log10(world_data_clean$population + 1)

# --------------------------------
# Step 18: Create target variable
# --------------------------------

median_gdp <- median(world_data_clean$gdp_per_capita, na.rm = TRUE)

world_data_clean$high_gdp_per_capita <- ifelse(
  world_data_clean$gdp_per_capita >= median_gdp,
  1,
  0
)

# --------------------------------
# Step 19: Final dataset check
# --------------------------------

print(head(world_data_clean))
print(dim(world_data_clean))
str(world_data_clean)
print(summary(world_data_clean))
print(table(world_data_clean$high_gdp_per_capita))

if (interactive()) {
  View(world_data_clean)
}

# --------------------------------
# Step 20: Save final dataset as CSV
# --------------------------------

write.csv(
  world_data_clean,
  "dataset/world_bank_gdp_project_dataset.csv",
  row.names = FALSE
)
```

```r
# ============================================================
# Exploratory Data Analysis
# ============================================================

plot1 <- ggplot(world_data_clean, aes(x = factor(high_gdp_per_capita))) +
  geom_bar() +
  labs(
    title = "High vs Low GDP Per Capita",
    x = "High GDP Per Capita",
    y = "Count"
  )

print(plot1)

ggsave(
  "plots/01_high_low_gdp_count.png",
  plot1,
  width = 7,
  height = 5
)

plot2 <- ggplot(world_data_clean, aes(x = gdp_per_capita)) +
  geom_histogram(bins = 30) +
  labs(
    title = "Distribution of GDP Per Capita",
    x = "GDP Per Capita",
    y = "Frequency"
  )

print(plot2)

ggsave(
  "plots/02_gdp_distribution.png",
  plot2,
  width = 7,
  height = 5
)

plot3 <- ggplot(world_data_clean, aes(x = life_expectancy, y = gdp_per_capita)) +
  geom_point() +
  labs(
    title = "Life Expectancy vs GDP Per Capita",
    x = "Life Expectancy",
    y = "GDP Per Capita"
  )

print(plot3)

ggsave(
  "plots/03_life_expectancy_vs_gdp.png",
  plot3,
  width = 7,
  height = 5
)

plot4 <- ggplot(world_data_clean, aes(x = internet_users, y = gdp_per_capita)) +
  geom_point() +
  labs(
    title = "Internet Users vs GDP Per Capita",
    x = "Internet Users Percentage",
    y = "GDP Per Capita"
  )

print(plot4)

ggsave(
  "plots/04_internet_users_vs_gdp.png",
  plot4,
  width = 7,
  height = 5
)

avg_gdp_year <- world_data_clean %>%
  dplyr::group_by(year) %>%
  dplyr::summarise(avg_gdp = mean(gdp_per_capita, na.rm = TRUE), .groups = "drop")
```

```r
plot5 <- ggplot(avg_gdp_year, aes(x = year, y = avg_gdp)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Average GDP Per Capita by Year",
    x = "Year",
    y = "Average GDP Per Capita"
  )

print(plot5)

ggsave(
  "plots/05_average_gdp_by_year.png",
  plot5,
  width = 7,
  height = 5
)

top_gdp_countries <- world_data_clean %>%
  dplyr::group_by(country) %>%
  dplyr::summarise(avg_gdp = mean(gdp_per_capita, na.rm = TRUE), .groups = "drop") %>%
  dplyr::arrange(desc(avg_gdp)) %>%
  dplyr::slice_head(n = 10)

plot6 <- ggplot(top_gdp_countries, aes(x = reorder(country, avg_gdp), y = avg_gdp)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Top 10 Countries by Average GDP Per Capita",
    x = "Country",
    y = "Average GDP Per Capita"
  )

print(plot6)

ggsave(
  "plots/06_top_10_gdp_countries.png",
  plot6,
  width = 8,
  height = 5
)

# ============================================================
# Model 1: Decision Tree Classification
# Model 2: Random Forest Classification
# ============================================================

# --------------------------------
# Step 21: Decision Tree model dataset
# --------------------------------

tree_model_data <- world_data_clean %>%
  dplyr::select(
    year,
    log_population,
    life_expectancy,
    internet_users,
    urban_population_pct,
    unemployment_rate,
    high_gdp_per_capita
  )

tree_model_data$high_gdp_per_capita <- as.factor(tree_model_data$high_gdp_per_capita)

if (interactive()) {
  View(tree_model_data)
}

# --------------------------------
# Step 22: Train-test split
# --------------------------------

set.seed(123)
```

```r
train_index <- sample(
  1:nrow(tree_model_data),
  size = floor(0.7 * nrow(tree_model_data))
)

tree_train_data <- tree_model_data[train_index, ]
tree_test_data <- tree_model_data[-train_index, ]

# ------------------------------
# Step 23: Train Decision Tree model
# ------------------------------

tree_model <- rpart(
  high_gdp_per_capita ~ .,
  data = tree_train_data,
  method = "class"
)

# ------------------------------
# Step 24: Plot Decision Tree
# ------------------------------

rpart.plot(tree_model)

png(
  "plots/07_decision_tree_model.png",
  width = 1200,
  height = 800
)

rpart.plot(
  tree_model,
  main = "Decision Tree Model"
)

dev.off()

# ------------------------------
# Step 25: Decision Tree prediction and evaluation
# ------------------------------

tree_predictions <- predict(
  tree_model,
  tree_test_data,
  type = "class"
)

tree_confusion_matrix <- table(
  Actual = tree_test_data$high_gdp_per_capita,
  Predicted = tree_predictions
)

print(tree_confusion_matrix)

tree_accuracy <- sum(diag(tree_confusion_matrix)) / sum(tree_confusion_matrix)

print(
  paste("Decision Tree Accuracy:", round(tree_accuracy, 4))
)

# ============================================================
# Random Forest Classification
# ============================================================

# ------------------------------
# Step 26: Random Forest model dataset
# region and income are included to improve accuracy.
# gdp_per_capita is NOT used as a feature.
# ------------------------------

rf_model_data <- world_data_clean %>%
  dplyr::select(
    year,
    region,
    income,
```

```
    log_population,
    life_expectancy,
    internet_users,
    urban_population_pct,
    unemployment_rate,
    high_gdp_per_capita
  )

rf_model_data$region <- as.factor(rf_model_data$region)
rf_model_data$income <- as.factor(rf_model_data$income)
rf_model_data$high_gdp_per_capita <- as.factor(rf_model_data$high_gdp_per_capita)

if (interactive()) {
  View(rf_model_data)
}

# --------------------------------
# Step 27: Use same train-test split
# --------------------------------

rf_train_data <- rf_model_data[train_index, ]
rf_test_data <- rf_model_data[-train_index, ]

# --------------------------------
# Step 28: Train Random Forest model
# --------------------------------

set.seed(123)

rf_model <- randomForest(
  high_gdp_per_capita ~ .,
  data = rf_train_data,
  ntree = 500,
  mtry = 3,
  importance = TRUE
)

print(rf_model)

# --------------------------------
# Step 29: Random Forest prediction and evaluation
# --------------------------------

rf_predictions <- predict(
  rf_model,
  newdata = rf_test_data,
  type = "class"
)

rf_confusion_matrix <- table(
  Actual = rf_test_data$high_gdp_per_capita,
  Predicted = rf_predictions
)

print(rf_confusion_matrix)

rf_accuracy <- sum(diag(rf_confusion_matrix)) / sum(rf_confusion_matrix)

print(
  paste("Random Forest Accuracy:", round(rf_accuracy, 4))
)

# --------------------------------
# Step 30: Random Forest feature importance
# --------------------------------

rf_importance <- importance(rf_model)

print(rf_importance)

rf_importance_df <- data.frame(
  Feature = rownames(rf_importance),
  MeanDecreaseGini = rf_importance[, "MeanDecreaseGini"],
  row.names = NULL
)
```

```r
rf_importance_df <- rf_importance_df %>%
  dplyr::arrange(desc(MeanDecreaseGini))

write.csv(
  rf_importance_df,
  "dataset/random_forest_feature_importance.csv",
  row.names = FALSE
)

png(
  "plots/08_random_forest_feature_importance.png",
  width = 1000,
  height = 700
)

varImpPlot(
  rf_model,
  main = "Random Forest Feature Importance"
)

dev.off()

# ============================================================
# Evaluation Metrics
# ============================================================

calculate_metrics <- function(conf_matrix) {

  conf_matrix <- as.matrix(conf_matrix)

  if (!all(c("0", "1") %in% rownames(conf_matrix)) || !all(c("0", "1") %in% colnames(conf_matrix))) {
    stop("Confusion matrix must contain both classes 0 and 1.")
  }

  TP <- conf_matrix["1", "1"]
  TN <- conf_matrix["0", "0"]
  FP <- conf_matrix["0", "1"]
  FN <- conf_matrix["1", "0"]

  accuracy <- (TP + TN) / sum(conf_matrix)
  precision <- ifelse((TP + FP) == 0, 0, TP / (TP + FP))
  recall <- ifelse((TP + FN) == 0, 0, TP / (TP + FN))
  f1_score <- ifelse((precision + recall) == 0, 0, 2 * ((precision * recall) / (precision + recall)))

  return(
    c(
      Accuracy = accuracy,
      Precision = precision,
      Recall = recall,
      F1_Score = f1_score
    )
  )
}

tree_metrics <- calculate_metrics(tree_confusion_matrix)
rf_metrics <- calculate_metrics(rf_confusion_matrix)

model_results <- data.frame(
  Model = c("Decision Tree", "Random Forest"),
  Accuracy = c(tree_metrics["Accuracy"], rf_metrics["Accuracy"]),
  Accuracy_Percentage = c(
    paste0(round(tree_metrics["Accuracy"] * 100, 2), "%"),
    paste0(round(rf_metrics["Accuracy"] * 100, 2), "%")
  ),
  Precision = c(tree_metrics["Precision"], rf_metrics["Precision"]),
  Recall = c(tree_metrics["Recall"], rf_metrics["Recall"]),
  F1_Score = c(tree_metrics["F1_Score"], rf_metrics["F1_Score"])
)

print(model_results)

# ============================================================
# Save model results
# ============================================================
```

```r
write.csv(
  model_results,
  "dataset/world_bank_model_results.csv",
  row.names = FALSE
)

write.csv(
  as.data.frame(tree_confusion_matrix),
  "dataset/decision_tree_confusion_matrix.csv",
  row.names = FALSE
)

write.csv(
  as.data.frame(rf_confusion_matrix),
  "dataset/random_forest_confusion_matrix.csv",
  row.names = FALSE
)

# ============================================================
# Final Output
# ============================================================

print("Project completed successfully using R-based web scraping/data extraction.")
print("Dataset saved as: dataset/world_bank_gdp_project_dataset.csv")
print("Model results saved as: dataset/world_bank_model_results.csv")
print("Decision Tree confusion matrix saved.")
print("Random Forest confusion matrix saved.")
print("Random Forest feature importance saved.")
print("Graphs saved inside: plots folder")
```